COMPILER CONSTRUCTION PROJECT

ROMAN URDU COMPILER

README AND USER MANUAL

Execution Modes	Interpreter: direct AST execution
	Compiler: LLVM IR generation for AOT compilation
Target Platform	Arch Linux

Submitted By

Name	Roll Number
Muhammad Faizan	23L-0567
Maahi Ibn Ahmar	23L-0990
Mahad Bin Atif	22L-4951

Table of Contents

1. Supported Language Features
2. Arch Linux Dependencies
3. Compile the Project
4. Interpreter Mode
5. AOT Compiler Mode
6. Language Example
7. Run the Automated Tests
8. Optional Docker Verification
9. Project Files
10. Troubleshooting
11. Clean Generated Files

About this document

This manual explains how to install the required software, build the unified project, use both execution modes, generate LLVM IR, create a native executable, and execute the supplied tests on Linux.

Commands were checked using the official Arch Linux container with current Flex, Bison, GCC, and Clang packages.

1. Supported Language Features

- Integer variables using `adad`
- Assignment and arithmetic: `+`, `-`, `*`, `/`, `%`
- Comparisons: `chota hay` (`<`), `chota barabar` (`<=`), `bara hay` (`>`), `bara barabar` (`>=`), `barabar` (`==`), and `nahibarabar` (`!=`)
- Boolean operators: `&&`, `||`, `!`
- Conditional blocks using `agar` and `warna`
- While loops using `jabtak`
- Integer and string output using `dikhao`
- Program return values using `wapis`
- Statement termination using `kardo`
- Line comments beginning with `#` or `//`

2. Arch Linux Dependencies

Arch Linux is the primary target platform. Install the required packages with:

```
sudo pacman -Syu --needed base-devel flex bison clang
```

Required tools:

- GCC or another C11 compiler for building the interpreter and compiler
- Flex 2.6 or newer for lexical analysis
- Bison 3.8 or newer for parsing
- Clang/LLVM for compiling generated LLVM IR
- GNU Make for the supplied build targets

No third-party C libraries are required.

3. Compile the Project

From the project root, run:

```
make clean
make
```

This creates:

- `bin/roman-urdu` - unified interpreter and LLVM IR compiler

Both modes use the same scanner, grammar, AST, and executable. Select the required behavior with a command:

```
./bin/roman-urdu interpret program.ru
./bin/roman-urdu compile program.ru output.ll
```

4. Interpreter Mode

Pass one Roman Urdu source file to the interpreter:

```
./bin/roman-urdu interpret examples/demo.ru
```

Expected output:

```
y bara hai
1 se 5 tak jama:
15
```

The interpreter returns the value supplied to `wapis`. A syntax error includes its source line number. Runtime errors, such as use of an undefined variable or division by zero, are written to standard error.

5. AOT Compiler Mode

Generate LLVM IR:

```
./bin/roman-urdu compile examples/demo.ru build/demo.ll
```

Compile the IR into a native executable:

```
clang build/demo.ll -o build/demo
```

Run the executable:

```
./build/demo
```

The result matches the interpreter output. If the output filename is omitted, the compiler writes LLVM IR to standard output:

```
./bin/roman-urdu compile examples/demo.ru > build/demo.ll
```

The complete three-command workflow is also available through:

```
make demo
```

6. Language Example

Roman Urdu source files conventionally use the `.ru` extension.

```
adad n = 5 kardo
adad factorial = 1 kardo

jabtak (n bara hay 1) {
    factorial = factorial * n kardo
    n = n - 1 kardo
}

agar (factorial barabar 120) {
    dikhao("jawab durust hai") kardo
} warna {
    dikhao("jawab ghalat hai") kardo
}

dikhao(factorial) kardo
wapis 0 kardo
```

Keyword reference:

Roman Urdu	Meaning
`adad`	Declare an integer variable
`agar`	If
`warna`	Else
`jabtak`	While
`dikhao`	Print a string or integer expression
`wapis`	Return from the program

Roman Urdu	Meaning
`kardo`	End a statement
`barabar`	Equal comparison (==)
`nahibarabar`	Not-equal comparison (!=)
`bara hay`	Greater-than comparison (>)
`chota hay`	Less-than comparison (<)
`bara barabar`	Greater-than-or-equal comparison (>=)
`chota barabar`	Less-than-or-equal comparison (<=)

Variables must be declared before they are used at runtime. This first version uses a single program-wide integer namespace; nested block scopes and user-defined functions are outside the project scope. Integer expressions use signed 32-bit LLVM values in the AOT compiler.

7. Run the Automated Tests

Build the unified program and run end-to-end tests for both modes:

```
make test
```

The test script checks the demo, factorial, and comparison programs in both modes. In compiler mode it generates LLVM IR, compiles the IR with Clang, runs the native binaries, and compares their output with the expected results.

Expected final line:

```
All interpreter and compiler tests passed.
```

8. Optional Docker Verification

If Docker is available, reproduce the tested Arch Linux environment with:

```
docker build -t roman-urdu-compiler .
docker run --rm roman-urdu-compiler
```

The image builds the project and runs all tests during image creation. Its default command runs the demo.

9. Project Files

- `src/lexer.l` - Shared Flex scanner for both execution modes.
- `src/parser.y` - Shared Bison grammar that builds the AST.
- `src/main.c` - Command-line entry point and mode selection.
- `include/ast.h` - AST node definitions and interpreter interface.
- `include/codegen.h` - LLVM IR code generator interface.
- `include/frontend.h` - Shared parser interface.
- `src/ast.c` - AST construction, cleanup, symbol table, and interpreter execution.
- `src/codegen.c` - LLVM IR emission for expressions, variables, output, branches, and loops.
- `examples/demo.ru` - Full demonstration of variables, branches, loops, strings, and arithmetic.
- `examples/factorial.ru` - Small factorial example.
- `examples/comparisons.ru` - Equality and inequality keyword example.

- `tests/run_tests.sh` - End-to-end regression tests for both modes.
- `Makefile` - Linux build, test, demo, and cleanup targets.
- `Dockerfile` - Reproducible Arch Linux build and test environment.
- `README.md` - Markdown version of the project guide.
- `README.pdf` - Submission-ready PDF version of the project guide.
- `Roman_Urdu_Compiler.pdf` - Original project idea document.
- `LICENSE` - MIT license.

Generated files are placed under `build/` and `bin/`; both directories can be removed safely with `make clean`.

10. Troubleshooting

- `flex: command not found` or `bison: command not found`: install the packages listed under Arch Linux Dependencies.
- `clang: command not found`: install it with `sudo pacman -S clang`.
- `line N: syntax error`: check for a missing `kardo`, parenthesis, or brace near the reported line.
- `runtime error: undefined variable`: declare the variable with `adad` before assigning or reading it.
- `Permission denied` when running a script: invoke it through `sh tests/run_tests.sh`; executable permission is not required in that form.

11. Clean Generated Files

```
make clean
```

This removes only generated `bin/` and `build/` directories. Source files and documentation remain unchanged.